

# Scheduling in Real-Time Systems

Meeting deadlines when time is part of correctness

## Definition

A **real-time system** is a system in which time is essential. The computer must respond to external events correctly and within a fixed amount of time.

## 1. Why Time Matters

### Main Idea

In ordinary systems, a correct answer may still be useful even if it arrives a little late. In real-time systems, a late answer may be just as bad as a wrong answer.

### Deadline Failure

The scheduler must make sure important computations finish before their deadlines. If the computation takes too long, the system may fail even if the final result is logically correct.

## 2. Examples of Real-Time Systems

| System                   | Timing Requirement                                                               |
|--------------------------|----------------------------------------------------------------------------------|
| Compact disc player      | Bits must be converted into music fast enough to avoid distorted sound.          |
| Hospital patient monitor | Sensor readings must be processed quickly enough to detect dangerous conditions. |
| Aircraft autopilot       | Control decisions must be made in time to keep the aircraft stable.              |
| Factory robot control    | Motor and sensor actions must happen at precise times.                           |

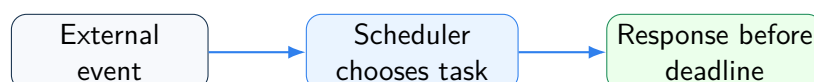
### Key Lesson

For real-time systems, correctness has two parts: producing the right answer and producing it before the deadline.

## 3. External Events and Responses

### Stimuli

Real-time systems usually react to stimuli generated by physical devices outside the computer.



## 4. Hard vs. Soft Real Time

| Type                  | Meaning                                                         | Consequence of Missing a Deadline                  |
|-----------------------|-----------------------------------------------------------------|----------------------------------------------------|
| <b>Hard real time</b> | Deadlines are absolute and must be met.                         | Failure may be unacceptable or dangerous.          |
| <b>Soft real time</b> | Deadlines are important but occasional misses can be tolerated. | Performance degrades, but the system may continue. |

### Hard Real-Time Rule

In a hard real-time system, missing a deadline is considered a system failure.

### Soft Real-Time Rule

In a soft real-time system, missing an occasional deadline is undesirable, but it may be acceptable.

## 5. Predictable Processes

### Known Behavior

Real-time behavior is usually achieved by dividing the program into processes or tasks whose behavior is predictable and known in advance.

### Short Tasks

These tasks are often short-lived and can run to completion in much less than a second. This makes it easier for the scheduler to reason about deadlines.

## 6. Periodic and Aperiodic Events

### Event Types

Real-time events can be classified by when they occur.

| Event Type       | Meaning                      | Example                                           |
|------------------|------------------------------|---------------------------------------------------|
| <b>Periodic</b>  | Occurs at regular intervals. | A sensor reading every 100 ms.                    |
| <b>Aperiodic</b> | Occurs unpredictably.        | An emergency button press or sudden fault signal. |

### Scheduler's Job

When an event is detected, the scheduler must run the needed process in time for the deadline to be met.

## 7. Schedulability

### Schedulable System

A real-time system is **schedulable** if the required tasks can actually be scheduled so all required deadlines are met.

### CPU Load Test

For  $m$  periodic events, if event  $i$  occurs every  $P_i$  seconds and needs  $C_i$  seconds of CPU time each time, then the total utilization is:

$$\sum_{i=1}^m \frac{C_i}{P_i}$$

### Schedulability Condition

The system can be handled only if:

$$\sum_{i=1}^m \frac{C_i}{P_i} \leq 1$$

## 8. Meaning of the Formula

| Symbol                 | Meaning                                                 |
|------------------------|---------------------------------------------------------|
| $m$                    | Number of periodic event streams.                       |
| $P_i$                  | Period of event $i$ , or how often it occurs.           |
| $C_i$                  | CPU time needed to handle one occurrence of event $i$ . |
| $\frac{C_i}{P_i}$      | Fraction of CPU capacity consumed by event $i$ .        |
| $\sum \frac{C_i}{P_i}$ | Total CPU utilization required by all periodic events.  |

### Impossible Load

If the sum is greater than 1, the tasks collectively demand more CPU time than the CPU can provide. Such a system cannot be scheduled under this simple load test.

## 9. Worked Example

### Three Periodic Events

A soft real-time system has three periodic events:

- Event 1: period 100 ms, CPU time 50 ms.
- Event 2: period 200 ms, CPU time 30 ms.
- Event 3: period 500 ms, CPU time 100 ms.

$$\frac{50}{100} + \frac{30}{200} + \frac{100}{500} = 0.50 + 0.15 + 0.20 = 0.85$$

#### Result

Since  $0.85 < 1$ , the system is schedulable by this utilization test. The events require 85% of the CPU, leaving about 15% spare capacity.



CPU utilization: 0.85 out of 1.00

## 10. Adding a Fourth Event

#### Remaining Capacity

The first three events use 0.85 of the CPU, so the remaining capacity is:

$$1.00 - 0.85 = 0.15$$

#### Fourth Event Limit

If a fourth event has period 1 second, its CPU time  $C_4$  must satisfy:

$$\frac{C_4}{1} \leq 0.15$$

So  $C_4$  can be at most 0.15 seconds, or 150 ms.

#### Result

The fourth event can be added only if it needs no more than 150 ms of CPU time per event.

## 11. Important Assumption

#### Ignored Overhead

The utilization calculation assumes context-switching overhead and other scheduler overheads are small enough to ignore.

#### Practical Meaning

Real systems may need extra safety margin because interrupts, context switches, cache effects, and OS overhead also consume time.

## 12. Static vs. Dynamic Scheduling

| Scheduling Type    | When Decisions Are Made                   | Requirement                                                      |
|--------------------|-------------------------------------------|------------------------------------------------------------------|
| Static scheduling  | Before the system starts running.         | Requires complete advance knowledge of tasks and deadlines.      |
| Dynamic scheduling | At run time, after execution has started. | Can react to changing conditions and less predictable workloads. |

### Static Scheduling

Static scheduling works only when the system has perfect information in advance about the work to be done and the deadlines to be met.

### Dynamic Scheduling

Dynamic scheduling makes decisions while the system is running, so it is more flexible when events or workloads are not fully known in advance.

## 13. Strengths and Challenges

| Aspect              | Explanation                                                       |
|---------------------|-------------------------------------------------------------------|
| Main goal           | Meet timing deadlines, not just maximize throughput or fairness.  |
| Key requirement     | Tasks must have predictable timing behavior.                      |
| Schedulability test | Checks whether periodic CPU demand is at most total CPU capacity. |
| Hard real-time risk | Missing one deadline may mean system failure.                     |
| Soft real-time risk | Occasional misses may be tolerable but reduce quality.            |

## 14. Big Picture

### Main Conclusion

Real-time scheduling is about completing work before deadlines. A system is schedulable only if the CPU can provide enough time for all required event handling. For periodic events, the basic utilization test is  $\sum C_i/P_i \leq 1$ .

### Exam Shortcut

Remember: hard real time means deadlines must not be missed. Soft real time means occasional misses are tolerable. For periodic events, compute:

$$\sum_{i=1}^m \frac{C_i}{P_i}$$

If the result is at most 1, the workload passes the basic schedulability test.

## 15. Quick Review

### Key Points

- In a real-time system, time is part of correctness.
- A correct result delivered too late may be useless.
- Examples include CD players, patient monitors, aircraft autopilots, and robot controllers.
- Hard real-time systems have deadlines that must be met.
- Soft real-time systems can tolerate occasional deadline misses.
- Real-time programs are often divided into predictable short tasks.
- Periodic events occur at regular intervals.
- Aperiodic events occur unpredictably.
- For periodic events, utilization is  $\sum C_i/P_i$ .
- The system is schedulable by the basic test if  $\sum C_i/P_i \leq 1$ .
- Static scheduling decides before execution begins.
- Dynamic scheduling decides at run time.